NANYANG TECHNOLOGICAL UNIVERSITY

USING LARGE LANGUAGE MODELS FOR QUERY OPTIMIZATION

Pugalia Aditya Kumar

College of Computing and Data Science

2025

NANYANG TECHNOLOGICAL UNIVERSITY

SCSE8338

USING LARGE LANGUAGE MODELS FOR QUERY OPTIMIZATION

Submitted in Partial Fulfilment of the Requirements
for the Degree of Bachelor of Engineering in Computer Science
of the Nanyang Technological University

by

Pugalia Aditya Kumar

College of Computing and Data Science

2025

Abstract

Query optimization remains a central yet complex challenge in database management systems due to the exponential growth in data and the explosion of possible query execution plan combinations. Traditional optimizers often rely on heuristics and statistics, which can lead to suboptimal plans, especially in the presence of data correlations and evolving workloads. In this work, we propose a novel pipeline that leverages Large Language Models (LLMs) for fine-grained query hint selection in PostgreSQL. Unlike previous approaches such as BAO, which apply a fixed hintset globally, our method generates localized hints for individual operators within a query execution plan, using the `pg_hint_plan` extension. The system comprises of two distinct pipelines: a Zero-Shot Pipeline, which directly prompts a Large Language Model (LLM) with query-specific information to generate operator-level hints without relying on prior examples; and a Multi-Shot Pipeline, which enhances hint generation using a Retrieval-Augmented Generation (RAG) approach that retrieves similar past query demonstrations to guide the LLM in producing more effective optimization hints. To implement the above two pipelines, the system integrates contrastive learning and Graph Neural Networks (GNNs) to create robust query embeddings, which are stored in a FAISS-based vector database to manage the storage and retrieval of demonstrations for improved queries. Our implemented system pipeline achieved an average query execution time reduction of 15% for all modified queries.

Acknowledgments

I would like to express my sincere gratitude to Prof. Cong Gao for his support and for providing the academic environment that made this project possible.

I am especially thankful to Li Zhaodonghui, a Ph.D. student under Prof. Gao, whose continuous support, insights, and availability at every stage of the project were invaluable. His patience in addressing my questions and his guidance in overcoming technical challenges played a crucial role in the successful completion of this work.

I would also like to express my gratitude to Nanyang Technological University for financially supporting certain expenses for this project, such as the OpenAI API credits which were necessary for the project's execution.

Contents

List of Tables

List of Figures

Chapter 1

Introduction

As the world tends more towards data-driven decision making, the volume of stored data continues to expand exponentially. With such an abundance of data, extracting the necessary information in a timely and efficient manner has become increasingly challenging, as query processing can possibly take hours for extremely large datasets. This makes query optimization crucial for all modern database management systems(DBMS) to ensure effective utilization of massive datasets across industries.

When a query is executed in a DBMS it is first parsed into a logical query tree which is a logical representation of the query's operations such as selection, projection, join, etc. without the physical execution details. Thereafter, a query optimizer converts the logical query plan to a physical query plan which specifies the exact algorithms and access methods to execute each query. It does so be applying algebraic transformation, join reordering, selecting optimal algorithms for each operation, etc. A logical query plan can have multiple equivalent physical query plans that can be executed to get the same output. Most modern day DBMS query optimizers generate all possible query plans and choose the one with the lowest estimated cost Fig[1.1]. However, as the number of relations in the query grows, the number of alternate query plans also grows exponentially, forcing the optimizer to rely on heuristics based on database statistics instead [2]. As a result, the query plan generated by the DBMS can often be suboptimal due to challenges in estimating the cost and cardinality of the query [3, 4].

Several studies have been conducted on improving cardinality estimation and cost modeling for query optimization [5, 6]. Earlier the problems of cardinality estimation were approached with

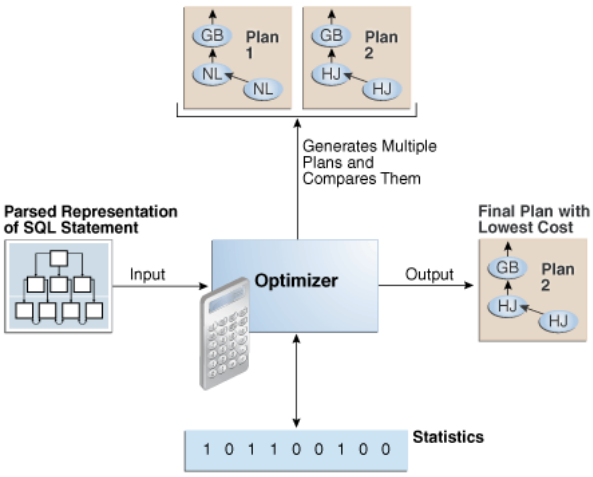

Figure 1.1: Query Optimization Process [1]

non-learning methods such as the use of histograms and sampling. In fact, most commercial DBMS use these non-learning based methods for cardinality estimation[7, 8, 9]. Despite advances in these techniques, they can still produce erroneous cardinality estimates as they fail to capture correlations between multiple attributes, and account for constantly changing data. [5, 3].

To overcome the shortcomings of the above non-learning-based methods, several research works have focused on using machine learning for query optimization. Bayesian belief networks have shown promise in capturing the correlation between attributes to improve cardinality estimation with joins [10, 11, 12]. However, learning the optimal structure of a Bayesian Network is computationally expensive. Additionally, representing conditional probability tables demands significant storage, especially for attributes with high domain cardinality. Multiple other neural network and deep reinforcement learning based frameworks have also been proposed, showing tremendous improvement [13, 14, 15, 16, 17, 18, 19]. Despite the promising results, these methods are not completely practical for one or more of the following reasons:

- They often require excessively long training times. Supervised learning-based cardinality estimators demand precise cardinality data, which is computationally expensive to gather, and reinforcement learning models needing training over thousands of queries before surpassing traditional optimizers.
- They can struggle to adapt to dynamic workloads and evolving datasets, necessitating frequent retraining, which is both time-consuming and impractical.

- They exhibit tail catastrophe, where although they may outperform traditional optimizers on average, they can cause severe performance degradation in rare cases when training data is sparse.
- The use of black-box deep learning models makes query optimization less transparent and harder to interpret compared to traditional cost-based optimizers. This limits the ability to fine tune the model to improve it further.
- the above models do not provide integration with commercial databases which can be an extensive task.

Marcus et al. [20] accurately identified these challenges and introduced BAO, a learned query optimizer integrated with PostgreSQL. BAO operates by providing query hints through the pg_hint_plan extension, enabling PostgreSQL's optimizer to select the most efficient query execution plan. It functions as an RL agent that dynamically activates or deactivates specific optimizer features (e.g. disable HashJoin) on a per-query basis to improve performance without replacing the traditional query optimizer. This allows it to enhance query optimization by leveraging an existing optimizer, rather than learning one from the scratch. BAO shows significant improvment on several Query optimization benchmarks and proves to be a practical solution that can be easily integrated with postgreSQL without the need for recompilation of the software. However, it also has certain limitations:

- BAO considers a very limited set of hints for each query, which means that it is not always able to learn the most optimal query plan for optimizations outside the hint set.
- BAO applies a single hint set to the entire query plan, which prevents it from optimizing individual sub-queries due to the high computational overhead. For example, while a hash join might be the best choice for one join operation, a nested loop could be more efficient for another. However, BAO cannot tailor optimizations at the operator level, as doing so would exponentially increase complexity.
- To learn from the past actions BAO only stores the k most recent actions to prevent unbounded growth of the experiences for BAO to learn from. This reduces the training overhead but can possibly reduce the model quality.

To address the above issues, this paper proposes a custom LLM pipeline designed to generate query hints for PostgreSQL. Previous work [21] on using Large Language Models for Query

Rewrite has shown promising results in improving efficiency, highlighting the potential for applying a similar concept for the current hint generation task. By redefining the current task from a contextual multi-armed bandit problem, as in [20], to a **rule-based generative task**, hints are no longer restricted to a set of Boolean values, such as enabling or disabling specific join or scan methods for the entire query. Instead, they can be applied more precisely to optimize each individual operator within the query. The Key contributions of this paper are:

- Generating a contrastive representation of queries using Graph Neural Networks for query by incorporating database statistics.
- Developing a a Retrieval Augmentation Generation(RAG) Framework which retrieves sample query improvements to provide as demonstrations to the LLM based on similarity and query efficiency enhancement.

Chapter 2

Preliminary

In this chapter, we first introduce some key concepts regarding cost-based optimizers and how query hints using pg_hint_plan work. Then, we formally define the problem for hint selection in section 2.2. Finally, we discuss some related work in section 2.3.

2.1 Query Optimization

Cost-based query optimizer: As discussed earlier cost-based optimizers select the most efficient query execution plan by estimating and minimizing resource costs. The process starts with **cardinality estimation**, where the optimizer uses the database statistics to predict the number of tuples processed at each stage of the query execution. Next, **cost modeling** is used to assign estimated computational costs (CPU, I/O, memory) to different execution strategies, such as scan or join algorithms. Finally, **plan enumeration** generates multiple equivalent query execution plans, evaluates their costs, and selects the query plan Q, with the lowest estimated cost. QP consists of a series of operation $o_1, o_2, ...o_n$

pg_hint_plan: pg_hint_plan overrides the optimizer's cost based plan selection and forces it select the plan based on user specified hints. The optimizer only enumerates the plans that adhere to the hints specified by the user and chooses the final execution plan based on it. The extension provides multiple different type of hints [22] primarily focused on modifying join and scan methods. Some examples can be seen in [Table 2.1].

Hint Group	Hint Format	Description
Scan method	`SeqScan(table)`	Forces a sequential scan on the table.
	`IndexScan(table[ index...])`	Forces an index scan on the table, restricting it to specified indexes if available.
	`NoIndexScan(table)`	Prevents the optimizer from using index and index-only scans on the table.
Join method	`NestLoop(table table[ table...])`	Forces a nested loop join for the specified tables.
	`HashJoin(table table[ table...])`	Forces a hash join for the specified tables in the query.

Table 2.1: pg_hint_plan hint examples

2.2 Hint Selection

With the introduction to Query plan tree and query hints, we formally define our problem as:

Definition 2.1: Consider PostgreSQL's query optimizer producing an initial query plan QP consisting of n operations. Given a set of all possible query hints $HS(QP) = \{H_1, H_2, \dots\}$, where $H_i = \{h_{i1}, h_{i2}, \dots, h_{in}\}$, and each h_{ij} is a valid hint for the corresponding operation o_j, such that the resulting query plan $QP' = \text{ApplyHints}(QP, H_i)$ is a valid query plan. The objective is to find a query hint H^* such that it minimizes the execution latency of the query plan

$$H* = \arg\min_{H} \text{Latency}(QP')$$

2.3 Related Work

2.3.1 Learned Query Hint Selection

Marcus et al. defined the problem of query hints selection as a contextual multi-armed bandit problem where each hint set represents an "arm" that can be chosen to optimize query execution. BAO builds on this formulation by integrating a tree convolutional neural network (TCNN) with Thompson sampling to enhance the optimizer's decision-making process. When a query is submitted, BAO first generates multiple query execution plans, each corresponding to a different hint set. Each plan is then transformed into a vectorized tree representation, which serves as input to BAO's TCNN model. The neural network estimates the expected execution performance of each plan based on its structure and optimizer-provided statistics. Instead of simply selecting the query plan with the lowest estimated cost, BAO applies Thompson sampling to balance exploration and exploitation. This allows the system to continuously refine its predictions by occasionally testing alternative hint sets, even when a promising option has already been identified. Once a plan is executed, the observed performance is fed back into the system, updating BAO's predictive model and improving future hint selections. By adopting this learned approach, BAO adapts to changes in query workloads, schema modifications, and variations in data distributions without requiring manual intervention. However, as discussed earlier, BAO applies a single hint set to the entire query, meaning it cannot optimize individual operators within a query plan. Furthermore, to learn which optimizer hints lead to better performance, BAO requires executing queries under different hint sets. Hence, in practice, BAO could have prohibitively expensive query execution overheads

Several models have been proposed that improve upon BAO's architecture to tackle the above issue. FASTGres [23] introduces a context-aware partitioning strategy coupled with supervised learning-based hint selection for subquery optimization. FASTGres groups queries into contexts based on their multi-way joins, ensuring that structurally similar queries are optimized together. This partitioning occurs at different levels of granularity: a coarse level, where all queries are treated as a single group; a table-based level, where queries are grouped by their joined tables; and a fine-grained level, where queries are grouped by both joined tables and join predicates. This divide-and-conquer approach allows FASTgres to learn more specialized hint selection strategies within each query context, improving accuracy compared to BAO's global model.

However, since FASTgres uses pretrained supervised model for hint selection it could show limited adaptability to unseen query types.

Autosteer [24] introduces an automated hint-set discovery mechanism, which greedily generates hint-sets per query. This removes the need for deep system-specific knowledge and enables AutoSteer to be easily integrated into various SQL databases, such as PostgreSQL, PrestoDB, SparkSQL, MySQL, and DuckDB. However, this includes additional overhead in hint generation which is further amplified by the sequential greedy algorithm which cannot be parellelized.

Another framework— DeepO introduced by Sun et al. [25] uses Bayesian Neural Networks (BNN) to generate hint with the lowest estimated cost. It uses a tree structured network to embed query execution plans into a hierarchical representation that preserves the relationships between operators. It collects execution logs of queries and their actual runtime and trains a BNN on the query embeddings and actual runtime to predict execution costs. When a new query arrives, DeepO generates multiple candidate query plans with different scan methods, join methods, and join orders, evaluates them using the BNN-based cost model, and ranks them based on their predicted execution costs and confidence intervals.

Recent work by Surgey and Surgey [26] introduced HERO, which improves query optimization by using an ensemble of context-aware models instead of neural networks, ensuring reliable and interpretable hint selection. It stores past query plans in a graph-based storage system, allowing it to reuse previous optimizations and speed up inference. Furthermore, to efficiently explore the hint space, it employs a budget-controlled local search algorithm, balancing exploration time with performance gains. Lastly, it integrates degree of parallelism control, optimizing both operator-related hints and execution parallelism, resulting in faster and more stable query performance. However, HERO similar to BAO, only focuses on coarse grained hints that apply to the entire query plan, hence, is limited by the search space.

2.3.2 LLMs for Query Optimzation

LLMs have recently been explored for query optimization, particularly for their strong performance in generative tasks such as rule-based query rewriting. Li et al. [21] introduce an LLM-based pipeline that incorporates a novel demonstration selector to accurately choose rules for rewriting SQL queries. The demonstration selector identifies a similar query from a pool of previously successful query rewrites to serve as a reference for the LLM, helping it make

better decisions when selecting rewrite rules. The paper highlights the importance of accurately selecting demonstrations for in-context learning in LLMs to increase the reliability of the model and provide efficient rewrite rules. This can be extended to our current problem where we select multiple demonstrations instead of a single one to help the LLM understand the hints that improve the query.

A recent study by Akioyamen et al. [27] demonstrated the potential of LLMs in selecting query hints effectively. Their model, LLMSteer used an LLM to embed a query into a dense feature vector which was then used to train a classifier for selecting hints. The study shows how a simple approach using LLMs can be used to improve query optimization, highlighting the potential for further exploration.

2.3.3 Embedding Queries

Generating accurate vector representations of queries is crucial for selecting meaningful demonstrations. Tree-RNNs and CNNs have been a popular choice used by several researches in the past [20, 14, 28, 29]. While these methods help to incorporate the hierarchical structure of the query tree along with information on the operators and other database statistics, they suffer from some inherent drawbacks. Tree RNNs similar to traditional RNNs cannot be effectively capture information from far away nodes in long query trees. Additionally, the lack of parallelization increases computation overhead. Tree CNNs on the other hand can only capture information of nearby nodes based on the filter size. To over come these issues QueryFormer [30], a tree-structured Transformer model was introduced by Zhou et al. It overcomes limitations of Tree-RNNs by using self-attention to model parent-child dependencies and long-range information flow efficiently. The model integrates height encoding to capture hierarchical relationships, tree-bias attention to restrict attention flow based on query plan structure, and a Super Node to aggregate global query information. Additionally, it incorporates database statistics like histograms, and sampling to enhance generalization. The paper shows the advantage of incorporating relevant statistics and using methods the capture the relationship between nodes for generating accurate embeddings.

Chapter 3

LLM Powered Query Hints Selector

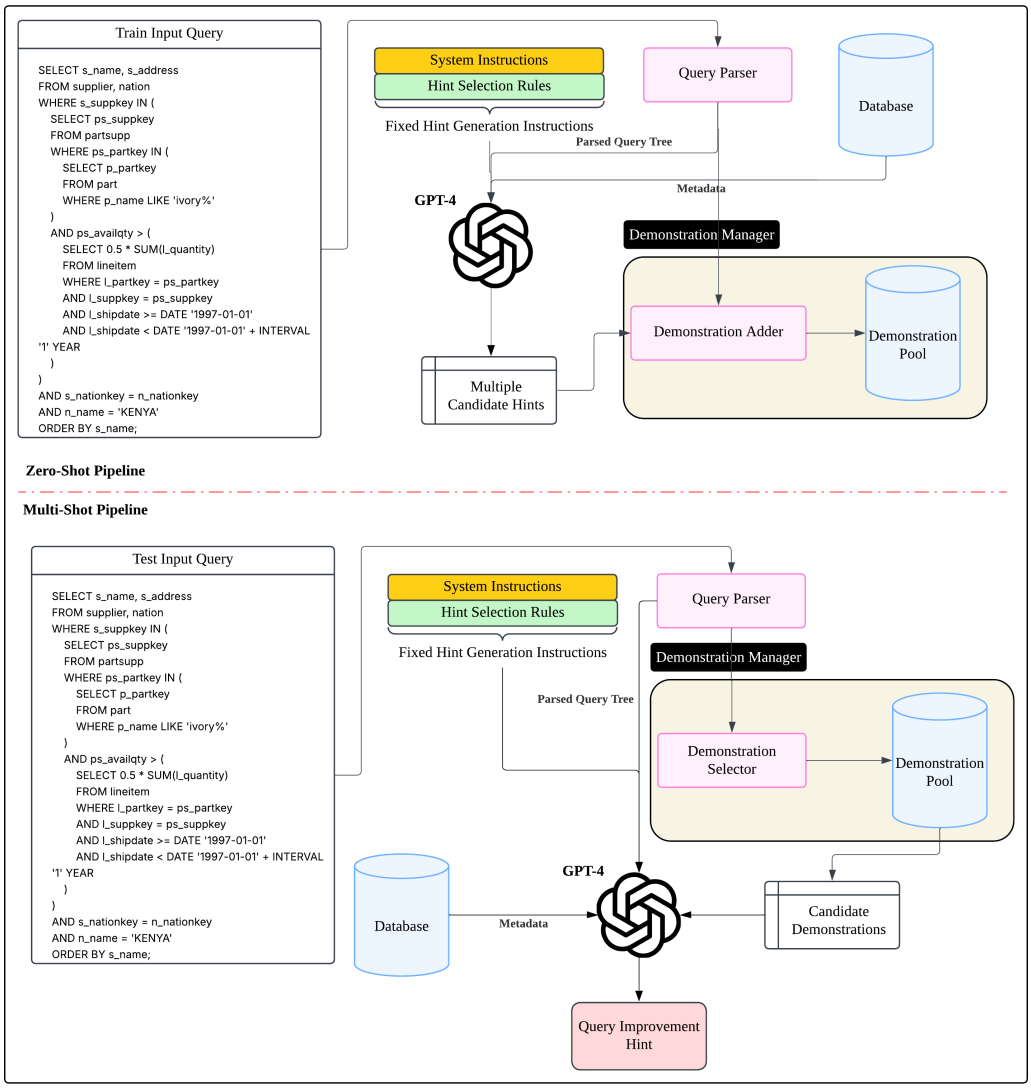

Figure 3.1: System Pipeline

In this chapter, we will describe in detail the implementation of the LLM Query Hints Selector pipeline, which leverages on In-Context Learning to enable the LLM to accurately select query improvement hints.

3.1 System Pipeline

Our proposed system pipeline for selecting hints using an LLM can primarily be divided into 2 phases as seen in Figure 3.1:

1. **The zero-shot pipeline**: In this phase, the LLM is prompted with the query and the relevant metadata to provide a set of k candidate hints to improve the query. From the set of k hints, those which improve the query are selected and added to the demonstration pool.

2. **The multi-shot pipeline**: This serves as the query optimizer. Given an input query, it retrieves relevant sets of hints and metadata from the demonstration pool created in the previous stage. These retrieved examples along with the user's input query are then used to prompt the LLM to generate optimized hints for improving query performance.

Formally, the objective of each stage can be defined as follows:

Definition 3.1: Given an LLM model M, a large enough set of n user queries $Q = \{q_1, q_2, \ldots, q_n\}$ and their parsed query plans $QP_i = \text{Parse}(q_i)$, a corresponding set of metadata, $MD = \{md_1, md_2, \ldots, md_n\}$ where m_i contains additional information regarding the database such as index, statistics, etc pertaining to the query q_i, and a set of rules for selecting hints L, the objective of the **zero shot pipeline** is to generate a set of k hints $Hints_i = \{H_1, H_2, \ldots, H_k\}$ for each query plan QP_i and add successful hints which reduce the query execution time to the Demonstration Pool D:

$$\forall (q_i, md_i) \in (Q, MD), \quad Hints_i = \text{Generate}(M, L, q_i, md_i), \quad Hints_i \subseteq HS(QP_i)$$

$$D = D \cup \{(H_j, QP_i) \mid \text{Latency}(\text{ApplyHints}(QP_i, H_j)) < \text{Latency}(QP_i), \quad \forall H_j \in Hints_i\}$$

Definition 3.2: Provided the Demonstration Pool D created in the previous stage, an unseen set of user queries $Q' = \{q'_1, q'_2, \ldots, q'_t\}$, and their corresponding set of query plans $QP'_i = \text{Parse}(q'_i)$

and metadata $MD' = \{md'_1, md'_2, \ldots, md'_t\}$, the objective of the multi-shot pipeline is to retrieve a set of top p candidate demonstrations CD_i for each query based on the criteria C and use it to generate optimal hints for the unseen test query.

$$\forall q'_i \in Q', \quad CD_i = \text{Retrieve}(D, C, QP'_i, p)$$

$$\forall (q'_i, md'_i) \in (Q', MD'), \quad H_i = \text{Generate}(M, L, q'_i, md'_i, CD_i)$$

This two-step approach ensures the accuracy of the output produced by the LLM through the use of In-Context Learning which allows us to overcome the issue of hallucination. The Zero-shot pipeline essentially functions as a trial-and-error mechanism to gather hints which can reduce the query execution time. Subsequently, for any unseen test queries, successfully stored hints of similar queries are retrieved and provided as demonstrations to the LLM allowing it to produce much more focused outputs.

Central to both of these pipelines is the Demonstration Manager which embeds, stores, and retrieves demonstrations with a focus on speed and accuracy to reduce the execution latency whilst maintaining a low search cost. Its implementation with regard to the zero-shot pipeline and the multi-shot pipeline is discussed in detail in the subsequent chapters.

Chapter 4

Zero Shot Pipeline

The workflow of the Zero-Shot Pipeline is as follows:

1. Extracting relevant metadata from the database, including indexes, table statistics, and database configurations.
2. Parsing the query into a query tree structure for better representation of its logical and physical execution.
3. Zero-Shot Prompting the LLM to get candidate hints.
4. Adding the original query, the hints that improved execution time, and other relevant information to the the demonstration pool using the demonstration manager. This requires:
    - Creating a Vector Representation of the original query tree.
    - Storing the vector representation using a vector database and linking it to the additional data stored separately.

In the following sections the above implementations will be discussed in detail.

4.1 Metadata retrieval from Database

Below you can find the description of the data regarding indexes, database configurations and table statistics which are retrieved from the database:

Column Name	Description
table_name	Name of the table to which the index belongs.
index_name	Name of the index.
column_names	Comma-separated list of columns indexed.
is_unique	Indicates whether the index enforces uniqueness
is_primary	Indicates whether the index is a primary key index

Table 4.1: Description of Index Metadata Columns

Column Name	Description
name	Name of the database configuration parameter.
setting	Current value of the parameter.
unit	Unit of measurement for the parameter (e.g., MB, ms).
context	Scope of the setting (e.g., postmaster, user, etc.).

Table 4.2: Description of Database Configuration Settings

Column Name	Description
schemaname	Name of the schema containing the table.
tablename	Name of the table for which statistics are retrieved.
column_name	Name of the column in the table.
null_frac	Fraction of rows in the column that are NULL.
avg_width	Average width (in bytes) of the column values.
n_distinct	Estimated number of distinct values in the column.
most_common_vals	Most common values in the column, if applicable.
most_common_freqs	Frequencies of the most common values.
histogram_bounds	Histogram of column values for distribution estimation.
correlation	Correlation coefficient between physical row ordering and column values.

Table 4.3: Description of Table Statistics Metadata

4.2 Parsing Query Tree

The first step to parse the the query into a query tree structure is to retrieve the query execution plan (QEP) from PostgreSQL by executing the query with the `'EXPLAIN (FORMAT JSON)'` command. This returns a hierarchical query execution plan with details such as node operations, cardinality, estimated cost, etc.

The retrieved QEP is then recursively parsed [Code Snippet 10.1] from the root node to the leaf node into a graph structure where node information is stored using the features shown in Table 4.4. The edges are stored in an array of shape $(2, e)$ where e is the number of edges. $edges[0]$ contain the index of the child node and $edges[1]$ contains the index of their parents. This is the sparse COO format for edge representation which is used for facilitating GNN processing using pytorch_geometric later.

Node Feature	**Description**
ID	Unique identifier for the node
Parent ID	Reference to the parent node
Node Type	Type of operation (e.g., Hash Join, Seq Scan)
Cumulative Cost	Total estimated execution cost up to this node
Planned Rows	Estimated number of rows processed
Relation Name	Table or relation involved in the operation
Index Name	Name of the index used, if applicable
Filter Condition	Predicate applied at this node, if any
Join Condition	Condition used for joins, if applicable
Index Condition	Condition used for index retrieval
Width	Estimated tuple width
Parent Hash	Boolean flag indicating if the parent is a hash operation

Table 4.4: Description of Features in Query Tree Nodes

4.3 Zero Shot Prompting

To get the candidate hints we initially prompt the LLM with just the general fixed instructions, the query plan, and the index information to return k different permutations of query plan hints. The fixed instructions define the tasks, rules for selecting hints, and the format of the output. The detailed structure of the prompt can be seen in Figure 4.1

The response from the LLM given the above prompt is parsed into k-different hints and appended in front of the original query to get k modified queries. These queries are then executed on

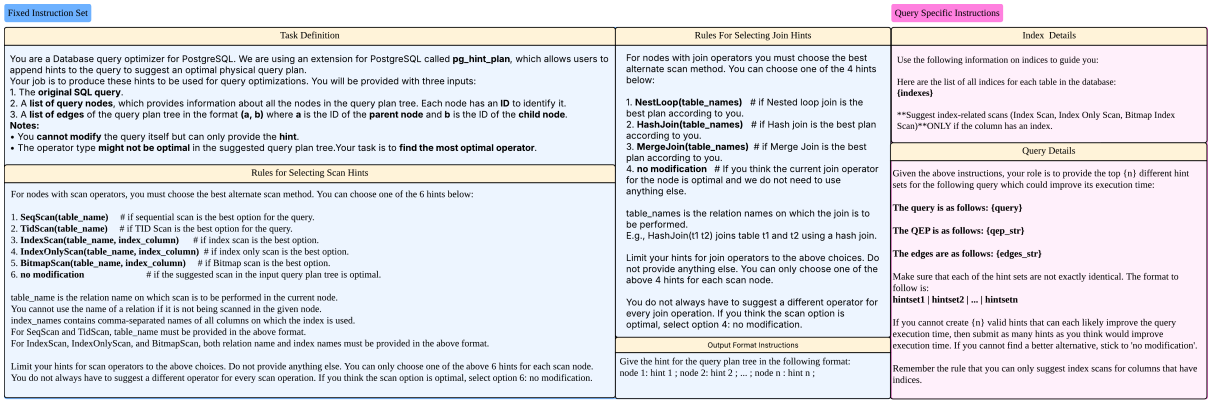

Figure 4.1: Zero Shot Prompt Structure

PostgreSQL and the execution time is compared to the execution time of the original query. The modified queries for which the execution time is less than the original query, the query-hint pair is added to the demonstration pool using the demonstration manager.

4.4 Demonstration Manager

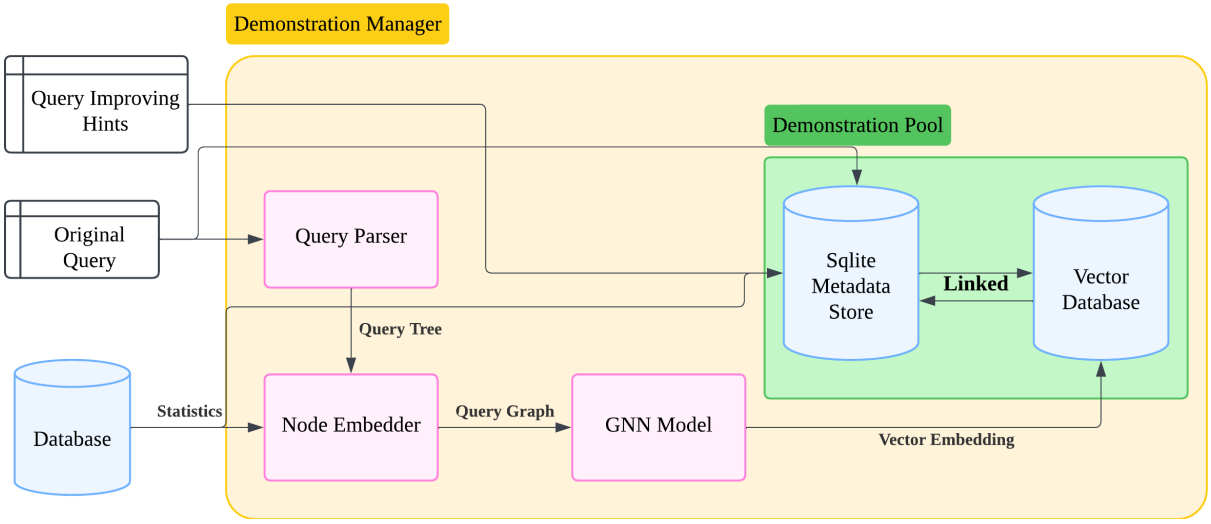

Figure 4.2: Outline of Demonstration Manager for Storage

The Figure 4.2 illustrates how the demonstration manager stores candidate demonstrations. The original query which is improved using LLM generated hints is parsed into a query tree [Section 4.2]. The nodes of the query tree are then converted into a vector representation using the node-embedder, representing the query tree as a vectorized graph. This graph is passed through a GNN model to compute a vector embedding of the entire query. The computed vector

embeddings of the original query is then stored in a vector database, and additional related data such as the hint which improved the query, percentage improvement, original query, and additional relevant database statistics are stored in an SQLite file and is linked to the vector embedding in the vector Database for fast retrieval.

Below, we will discuss in detail the implementation of the node-embedder, GNN model and the demonstration pool comprising of the vector database and the SQLite storage.

4.4.1 Creating Node Embeddings

Table 4.4 shows the node features obtained after parsing the query into a query tree. To create the vector representation of each node, the features are divided into two categories—textual and numerical—so they can be encoded appropriately.

Textual features such as labels, filters, and join/index conditions are encoded using **Bidirectional Encoder Representations from Transformers** (BERT). BERT is a large pre-trained, transformer-based language model that reads text in both directions—left-to-right and right-to-left—allowing it to capture deep bidirectional contextual representations of words based on their surrounding content. These context-aware representations are useful for capturing the semantic meaning of complex filters and conditions in query tree nodes, especially when multiple stacked or nested conditions are present. Previous research [20, 21, 25, 14] has predominantly relied on one-hot encoding for labels representing the operator type. While this method is valid given the categorical nature of the data, it inherently lacks the ability to capture similarity between different operators, as one-hot vectors are orthogonal. For instance, a 'hash join' node is more similar to a 'merge join' node than to a 'seq scan' node. Given this limitation, BERT embeddings were considered a more effective and semantically meaningful way to encode labels.

The numerical features—*cumulative cost*, *planned rows*, and *row width*—along with database statistics, are used to compute various **machine-independent ratios** that ensure the representations are generalizable across systems with varying memory and hardware configurations. The cumulative cost at each node is normalized and included as one of the features. The number of planned rows is multiplied by the row width to estimate the *output size* of each node. The *planned row ratio* is calculated as

$$planned\_row\_ratio = \frac{planned\_rows \times row\_width}{shared\_buffer\_size}.$$

Shared buffers in PostgreSQL act as a memory cache for storing data pages that are read from or written to disk. Thus, the planned row ratio indicates how large the output of a node is relative to what PostgreSQL can cache in memory for fast access. Additionally, for in-memory operations such as sorting, hashing, and aggregation, PostgreSQL allocates *working memory*. To capture this, we compute a *working memory ratio* as

$$work\_mem\_ratio = \frac{node\_input\_data\_size}{work\_mem}.$$

For hash operations, the effective working memory in PostgreSQL is increased by a parameter called *hash_mem_multiplier*, which is incorporated into the computation of the working memory ratio. By using these normalized ratios instead of raw cardinality estimates, our vector representation becomes **resource-aware**, allowing it to remain relevant and comparable across systems with different database configurations.

The final vector representation is concatenated using the above textual and numerical encodings as seen below:

```
# Combine all extracted features into a single matrix
features = np.column_stack((
    label_embeddings,               # (n_nodes, embedding_dim)
    cumulative_costs.reshape(-1, 1),
    planned_row_ratios.reshape(-1, 1),
    join_condition_embedding,       # (n_nodes, embedding_dim)
    index_condition_embedding,      # (n_nodes, embedding_dim)
    filter_embeddings,              # (n_nodes, embedding_dim)
    work_mem_ratios.reshape(-1, 1)
))
```

Code Snippet 4.1: Vector Representation

Given that the dimension of the BERT embeddings is 768, the resulting vector representation has a total dimensionality of 3075. This high dimensionality can make learning query representations using a Graph Neural Network (GNN) both computationally expensive and potentially less accurate. To address this, the dimensionality is reduced using Principal Component Analysis (PCA) while preserving 99% of the variance in the data. The final vector representation of each node has a dimension of **44**

4.4.2 GNN Model with Contrastive Learning

To learn robust representations of query execution plans, we employ a Graph Neural Network (GNN) trained using a contrastive learning objective. The key idea of a GNN is to learn a representative graph embedding by incorporating the information in the neighboring nodes. Prior to training, each query plan is parsed into a tree structure and represented as a graph $G = (V, E)$ as discussed in the previous sections.

Model Architecture. We use a multi-layer Graph Convolutional Network (GCN) [Code Snippet 10.2] to encode the graph structure. Unlike traditional Convolutional Neural Networks (CNN) which apply the convolution operation over a structured grid-data, in GCNs the convolution operation is generalized to work on irregular graph structures. Instead of a fixed grid, each node aggregates information from its immediate neighbors in the graph. This can be viewed as a form of message passing, where a node gathers and transforms features from its local neighborhood [31]. At each layer, the node embeddings are updated using information from their neighbors as follows:

$$H^{(l+1)} = f\left(\hat{D}^{-1/2}\hat{A}\hat{D}^{-1/2}H^{(l)}W^{(l)}\right)$$

Here, $\hat{A} = A + I$ is the adjacency matrix with added self-loops, $\hat{D}$ is the degree matrix of $\hat{A}$, $H^{(l)}$ represents node embeddings at layer l, $W^{(l)}$ is a learnable weight matrix, and f is the non-linear activation function ReLU. As seen in the above equation, information from further neighbors can be incorporated by stacking GCN layers. The final graph-level embedding z is obtained by applying a global sum pooling operation on the final node embeddings:

$$z = \text{Pool}(H^{(L)})$$

Data Preprocessing. Before passing the graph through the GCN, we augment each node's feature vector with *Laplacian Eigenvector Positional Encodings*. These encodings are derived from the eigenvectors of the graph's Laplacian matrix, $L = D - A$ which capture the global structural position of each node within the graph [32]. Specifically, we compute the k smallest non-trivial eigenvectors of the normalized Laplacian and use them as additional features, where k is a hyperparameter that can be set by the user. If the number of nodes in the query tree

is smaller than k, zero-padding is applied to maintain a consistent input size. The resulting positional encodings are concatenated with the original node features, enabling the GCN to incorporate not just local neighborhood information but also global topological context during message passing.

Contrastive Learning Objective. To train the model without labels, we use a contrastive objective based on the InfoNCE loss, similar to approaches such as SimCLR [33] and InfoGraph [34]. For each query graph G, two augmented views G_1 and G_2 are generated using stochastic transformations like node feature masking. These are encoded using the GNN to obtain embeddings z_1 and z_2.The contrastive loss objective encourages the embeddings of the two views of the same graph (positive pair) to be similar, while discouraging similarity with embeddings of other graphs (negative pairs). The InfoNCE loss for a sample i is defined as:

$$\mathcal{L}_i = -\log \frac{\exp(\text{sim}(z_1[i], z_2[i])/\tau)}{\sum_{j=1}^{N} \exp(\text{sim}(z_1[i], z_2[j])/\tau)}$$

where $\text{sim}(z_1, z_2)$ is the cosine similarity, τ is a temperature parameter, and N is the batch size. The final loss is averaged across all samples:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}_i$$

Contrastive learning has proven effective in capturing meaningful graph-level representations, especially in unsupervised or pretraining settings [35, 36]. Moreover, by learning from augmented versions of query plans, the model becomes robust to structural and feature-level variations.

4.4.3 Demonstration Pool

The demonstration pool consists of two components— the vector database to store the vector embedding of the query and an SQLite database for storing the additional information corresponding to that vector embedding.

Vector Database. The vector database is implemented using Facebook AI Similarity Search (FAISS) Library. FAISS stores vector data in contiguous blocks and optimizes search time using indices and GPU acceleration. FAISS provide multiple indexing options such as:

- IndexFlat: Performs exact nearest neighbor search by exhaustively comparing the query to every vector stored in a flat array.
- IndexIVFFlat: Speeds up search by clustering the vector space and limiting comparisons to vectors in the closest clusters using k-means. Each new vector is added to the nearest centroid.
- IndexHNSW(Hierarchical Navigable Small World): It builds a graph-based index where each vector is a node connected to its nearest neighbors, forming a navigable small-world graph. During search, the algorithm starts from an entry point and navigates the graph by greedily moving to neighbors that are closer to the query.

While IndexIVFFlat and and IndexHNSW are much faster at getting searching they are not a 100% accurate in finding. Moreover since our demonstration pool is not very large we decided to use IndexFlatIP which performs a bruteforce search over all the vectors and inner-product as the similarity metric. However, if the demonstration pool grows large, the index can be converted to IndexHNSW to trade off a slight fall in accuracy for much faster search. The FAISS index only stores vectors of a fixed length, hence, it is initialized with vector dimensions of the output dimension of the GNN Model as follows:

```
#initialise FAISS index.
if metric == "L2":
    self.index = faiss.IndexFlatL2(dimension)
elif metric == "cosine":
    self.index = faiss.IndexFlatIP(dimension)  # Cosine uses Inner Product
else:
    raise ValueError("Unsupported metric. Use 'L2' or 'cosine'.")

# Load existing FAISS index if available
try:
    self.index = faiss.read_index(index_path)
except:
    pass  # If the index file does not exist, it starts fresh
```

Code Snippet 4.2: Initialising the Vector Database

SQLite Database. SQLite is a lightweight, serverless relational database engine that stores the entire database in a single file on disk, which makes data retrieval significantly faster for small/medium sized data. This makes it ideal for our use case as we are just storing some

metadata related to the query vector which are to be accessed only via indices. The SQLite Database is initialized along with the FAISS index using the user provided column names:

```
def _init_sqlite(self):
    """Initializes SQLite database with user-defined metadata columns."""
    conn = sqlite3.connect(self.storage_path)
    cursor = conn.cursor()

    # Construct table schema dynamically
    columns_schema = ", ".join([f"{col} TEXT" for col in self.
   metadata_columns])
    cursor.execute(f"""
        CREATE TABLE IF NOT EXISTS metadata (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            {columns_schema}
        )
    """)

    conn.commit()
    conn.close()
```

Code Snippet 4.3: Initialising the SQLite Database for vectore metadata

FAISS index assigns a unique auto-incrementing index id to each vector that is added. Hence, the primary key id is set to autoincrement in the SQLite database so that each vector and its corresponding metadata that are added to the SQLite database have the same id. This makes retrieval via index numbers much faster.

Adding Demonstrations

Each demonstration that is added to the demonstration pool (Vector Database + SQLite Database) consists of the following:

- query vector: the vector representation of the original query that was improved.
- query: the original query
- hint: the hint retrieved from the LLM that reduced the query execution time.
- time_improved: the percentage reduction in query execution time

- database_config: Database configuration.

The query vector is added to the FAISS index and the rest are added to the SQLite database as metadata. FAISS does not natively support cosine similarity as a distance metric. Instead, it supports inner product (`IndexFlatIP`) and Euclidean distance (`IndexFlatL2`) by default. To approximate cosine similarity, the query vectors are **L2-normalized** before being added to the index. This follows from the proof below:

Given two vectors $\vec{u}$ and $\vec{v}$, the cosine similarity is defined as:

$$\text{cosine\_similarity}(\vec{u}, \vec{v}) = \frac{\vec{u} \cdot \vec{v}}{\|\vec{u}\| \cdot \|\vec{v}\|}$$

If both vectors are normalized to unit length:

$$\|\vec{u}\| = \|\vec{v}\| = 1$$

Then the cosine similarity reduces to the inner product:

$$\vec{u} \cdot \vec{v} = \text{cosine\_similarity}(\vec{u}, \vec{v})$$

Hence, to perform cosine similarity search using FAISS, vectors are normalized before insertion and querying. This enables the use of `IndexFlatIP` (inner product index) as a proxy for cosine similarity search.

After the vector is added to the FAISS index, its corresponding metadata is added to the SQLite database, completing the process of adding a demonstration. The full implementation can be seen below:

```
def add_vectors(self, texts: Optional[List[str]] = None, embeddings :
    Optional[List[List]] = None, metadata: pd.DataFrame = None):
    """
    Adds a batch of text vectors to the FAISS index and stores metadata.

    :param texts: List of text data to be embedded and stored.
    :param embeddings: List of precomputed embeddings to be stored.
    :param metadata: Optional list of metadata related to each text.
```

```
    """
    if embeddings is not None:
        pass
    elif texts is not None and self.model is not None:
        embeddings = self.model.encode(texts, convert_to_numpy=True).astype("float32")
    else:
        raise Exception('Either No text/model or embeddings not provided.')


    # Normalize for cosine similarity
    if self.metric == "cosine":
        if not embeddings.flags['C_CONTIGUOUS']:
            # Ensure embeddings are float32 and C-contiguous
            embeddings = np.ascontiguousarray(embeddings, dtype=np.float32)
        faiss.normalize_L2(embeddings)

    # Add to FAISS index
    self.index.add(embeddings)
    faiss.write_index(self.index, self.index_path)  # Save index

    # Store metadata
    if self.storage_path and (metadata is not None):
        conn = sqlite3.connect(self.storage_path)
        cursor = conn.cursor()
        placeholders = ", ".join(["?" for _ in self.metadata_columns])
        column_names = ", ".join(self.metadata_columns)

        metadata_values = metadata[self.metadata_columns].fillna("").values.tolist()
        cursor.executemany(f"INSERT INTO metadata ({column_names}) VALUES ({placeholders})", metadata_values)
        conn.commit()
        conn.close()
    print('number of indices: ', self.index.ntotal)
```

Code Snippet 4.4: adding demonstrations

Chapter 5

Multi-Shot Pipeline

After the Zero-Shot pipeline completes execution to populate the demonstration pool, the multi-shot pipeline is used to test the effectiveness of the collected demonstrations to improve LLM hint suggestion on unseen queries. The workflow of the multi-shot pipeline is as follows:

1. Search the demonstration pool to retrieve k similar improved query demonstrations
2. Prompt the LLM with the included demonstrations to generate one optimal hintset
3. evaluate the modified query (hint + query) and store the results.

5.1 Retrieving Demonstrations

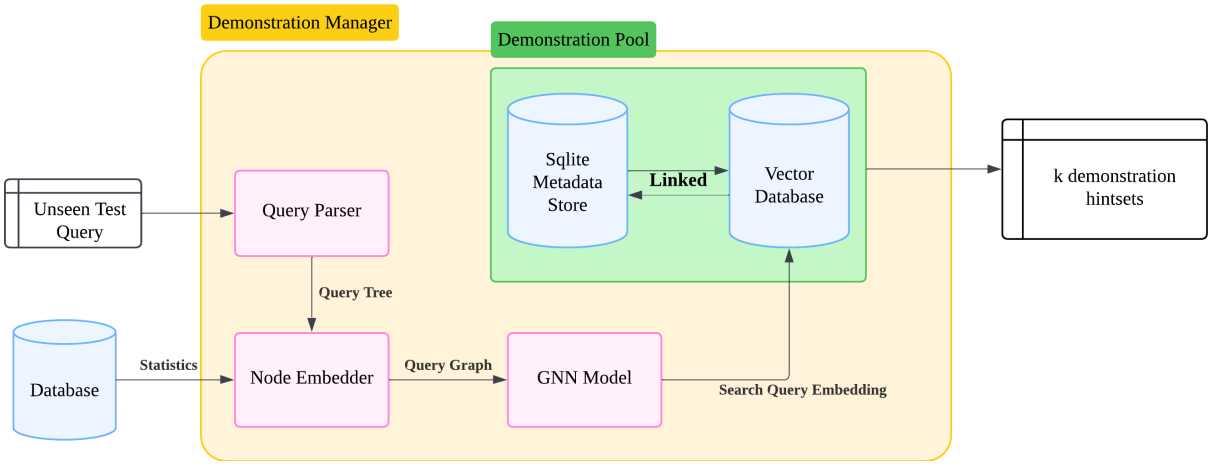

Figure 5.1: Retrieving Demonstrations

As seen in the above figure, the demonstration manager is used to retrieve k candidate hints to add to the prompt. The first step is to embed the test query into a vector embedding using methods discussed in the previous section. The embedded query is then normalized (to proxy cosine similarity using Inner Product) and used to search the FAISS vector database for queries with a similarity threshold provided by the user. This search is implemented using the `range_search` function in FAISS which returns `(distances, indices)`, where `distances` contain the similarity of all the vectors with cosine similarity greater than the threshold, and `indices` contain the corresponding index of those vectors.

The indices are then used to query the SQLite database to retrieve the metadata corresponding to the retrieved query vectors. The metadata forms the candidate demonstrations. If the number of candidate demonstrations based on cosine similarity returned is greater than k then, the top k demonstrations which achieved the highest query time reduction are returned. This approach ensures that the demonstrations are pertaining to previously improved queries which are very similar to the test query and achieved the maximum improvement.

5.2 Multi-Shot Prompting

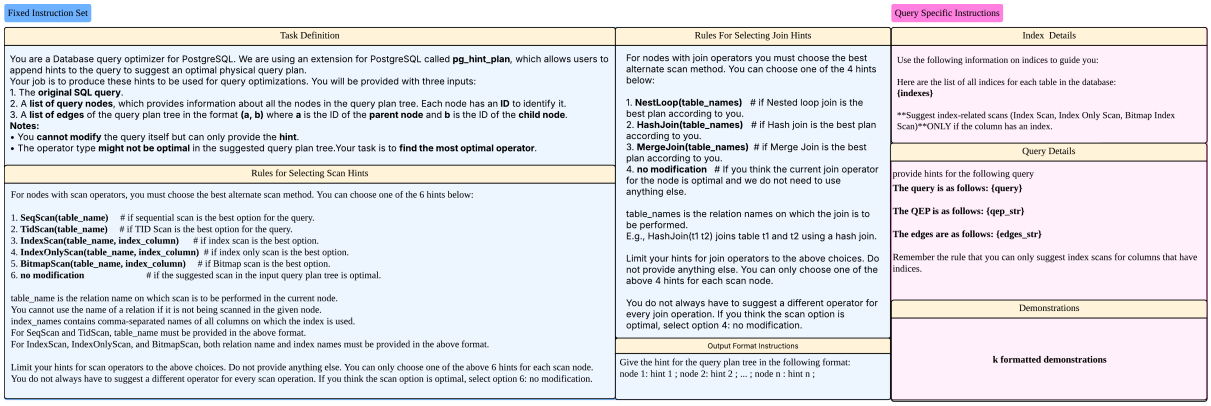

Figure 5.2: multi-shot prompt structure

The multi-shot prompt structure can be seen in the figure above. The query specific instruction now include demonstrations formatted using the candidate demonstrations retrieved from the demonstration pool. Each demonstration is in the following format:

```
{
```

```
  "role": "user",
  "content": "provide hints for the following query
            The query is as follows:
            <SQL_QUERY>
            The qep is as follows:
            <QEP_STRING>
            The edges are as follows:
            <EDGES_STRING>"
},
{
  "role": "assistant",
  "content": "<HINT_RESPONSE>"
}
```

Code Snippet 5.1: Demonstration Format for Hint Generation

In the demonstration format, the "user" role provides a structured query-specific prompt including the SQL query, its QEP, and edges, while the "assistant" role contains the expected model-generated hint response. This structure mimics a realistic user interaction to guide the LLM during multi-shot prompting.

Finally the hintset generated by the LLM is appended to the original query and executed in the database. The statistics on query improvement are computed and reported.

Chapter 6

Experiment and Results

6.1 Dataset

To test our developed system, we installed the `TPCH10` dataset on PostgreSQL in the server. `TPCH10` is a benchmark dataset, simulating a sales system with a 10 GB dataset, often used to test the performance of database systems. The dataset consists of 8 tables describing business sales data. The schema of the database looks as follows:

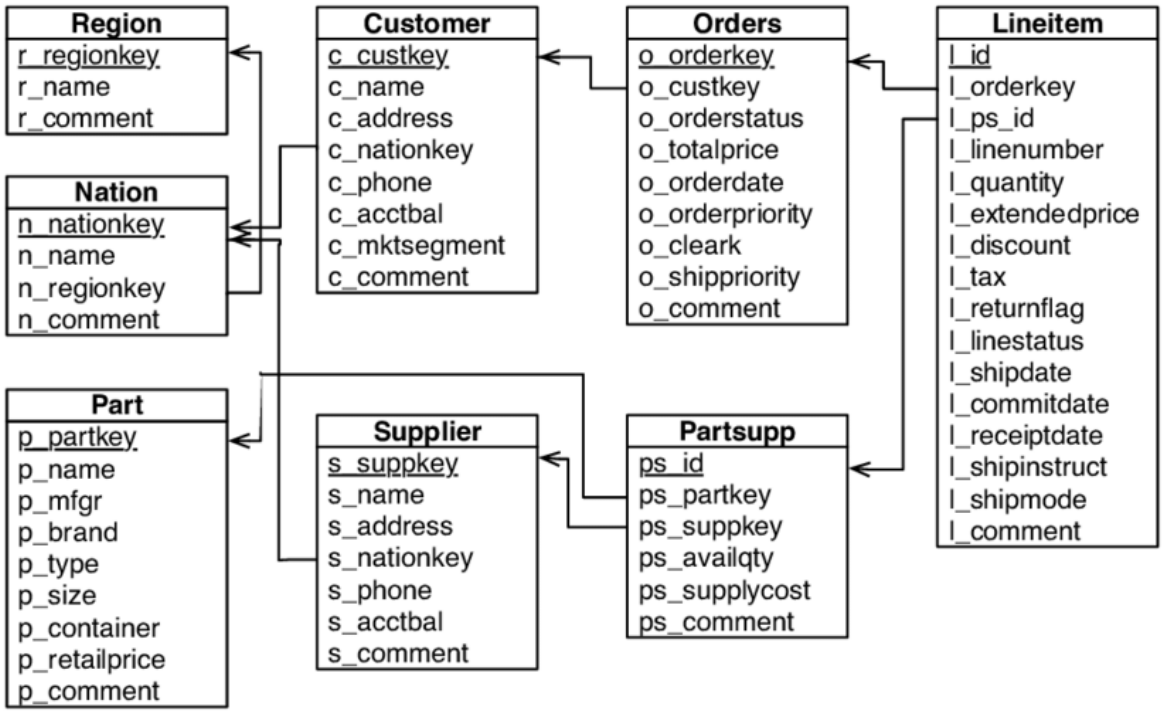

Figure 6.1: TPCH Database Schema

The train and test queries for the TPC-H benchmark were generated using the official TPC-H query templates by varying their parameters through the qgen utility provided in the TPC-H toolkit. Each query template was instantiated multiple times to produce diverse and representative query instances. A total of 4274 queries were created, comprising 3774 training queries and 500 testing queries.

6.2 LLM

The LLM used for this project is GPT-4o (2024-08-06), OpenAI's latest multimodal language model that is optimized for both performance and efficiency. GPT-4o was chosen for its enhanced reasoning capabilities, lower latency, and strong performance on structured text generation tasks, which are necessary for query rewriting and hint generation in SQL optimization. We selected this model due to its high accuracy and significantly lower hallucination rates when compared to other GPT variants and open-source models such as LLaMA during preliminary testing.

6.3 The Experiment

Model Training and Set up. The train dataset that we created was used to construct the demonstration pool using the zero-shot pipeline. Before running the process, all the database metadata was retrieved as discussed in section 4.1. Following that al the queries were parsed into a query tree and embedded into a large vectors of **3075** dimensions using the process discussed in the section 4.4.1. The large vectors were then trained to fit the PCA model to retain 99% variance, resulting in the vector dimensions reducing to **44**. The fitted PCA model was saved to be used during testing. After deriving the dense vector representation for the nodes of the query trees in the train dataset, the graph representation of the queries were used to train the GNN Model using contrastive learning loss (as described in section 4.4.2) to get a dense vector representation for the whole query tree of **64** dimensions. The model was trained using using mini-batches and Adam Optimizer. Early stopping was implemented with a patience of 3 to prevent overfitting on training data. The model hyperparameters are as follows:

Table 6.1: GNN Model Hyperparameters

Hyperparameter	**Value**	**Description**
Number of GNN Layers	2	Controls the number of message-passing steps, i.e., determines how far information is aggregated in the graph.
Hidden Layer Size	128	Dimensionality of the hidden GCN layer.
Output Embedding Size	64	Final output size of graph-level embedding.
Positional Encoding Dimension (k)	3	Number of Laplacian eigenvectors used for positional encoding of nodes.
Feature Masking Proportion	0.2	Proportion of node features randomly masked for data augmentation during contrastive learning.
Learning Rate	0.001	Step size for the Adam optimizer.
Batch Size	32	Number of graphs processed in each training iteration.
Temperature (τ)	0.5	Temperature used in contrastive InfoNCE loss.
Epochs	100	Number of training iterations over the dataset.

The trained model weights were again stored to be used during testing. With all our models learned and the data set up, we begin the experimentation.

Running the Experiment. Using the train queries and the trained models, we run the zero-shot pipeline as described in section 4. The LLM (ChatGPT) was prompted to generate 3 different hintsets (This number was chosen after a few trials to balance the run-time and generating sufficient demonstrations) for each train query. The hints that improved the query were added to the demonstration pool along with other metadata as described earlier. With the demonstration pool constructed, the test dataset was used to evaluate the performance of our system by running the multi-shot pipeline. For each test query, 3 candidate demonstrations were retrieved based on a similarity threshold of 0.95 to guide the LLM in generating optimization hints. The execution time of the modified query (hints + query) was compared with the original query, and the results were reported. For both training and testing, queries with a run time of more than 5 min were ignored in the experiment.

6.4 Results

After running the zero-shot pipeline using the 3774 training queries, we collected 1664 demonstrations. The average time improvement for the demonstrations was 13%, indicating that the pipeline was able to generate sufficiently large number of hints that improved the query execution time.

Finally, to evaluate the effectiveness of our hint selection mechanism, we analyzed the test results over 500 unseen queries using the multi-shot pipeline. Again, queries with run time greater than 5 minutes were ignored for the purpose of experimentation. Resulting in 372 test queries. Out of the 372 test queries, the LLM generated hints for 84 queries (22.6%), while 'no modification' was suggested for the rest, i.e, the LLM estimated that PostgreSQL's original query plan was optimal. Among these, **28 queries (33.3%)** showed improvement in execution time, while **56 queries (66.7%)** performed worse after modification.

The following plot summarizes the number of queries that improved versus those that degraded, as well as the average time savings or losses.

Figure 6.2: Left: Number of improved vs worse queries. Right: Average time improvement/degradation.

The above illustration shows that even though, only few queries were improved by the the model, the improvement time of those queries were significantly larger than that of the ones which where degraded.

The average run-time of the queries is compared and shown below, indicating that overall the query run-times were improved using our method.

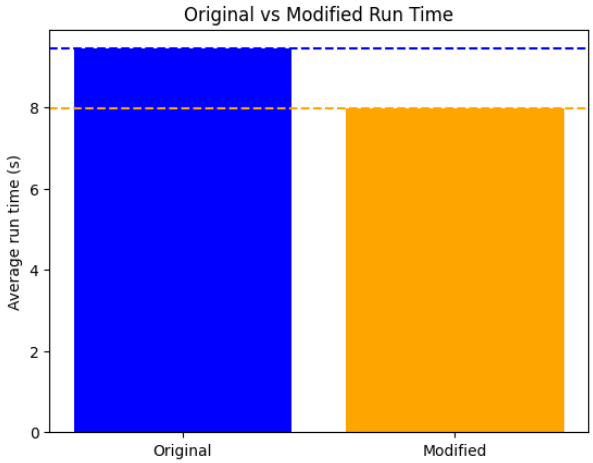

Figure 6.3: Average Query execution time for original vs modified queries

We further summarize the key performance metrics in Table 6.2.

Metric	Value
Total Test Queries	372
Queries with Hints Applied	84
Improved Queries	28
Degraded Queries	56
Average Improvement (Improved Queries)	7.67 seconds
Average Degradation (Degraded Queries)	1.65 seconds
Average Original Query Execution time	9.44 seconds
Average Modified Query Execution time	7.98 seconds
Overall Runtime Reduction	1.46 seconds
Percent Improvement (Modified Queries)	15.44%

Table 6.2: Summary of performance metrics on test queries.

These results demonstrate that, while LLM-based hint generation can yield significant performance gains in certain cases, it is still subject to potential degradation in others, indicating the need for refinement in demonstration retrieval and hint selection mechanisms. Moreover, since only 22.6% of the queries were modified during the testing, it suggests the need for further exploration during the zero-shot pipeline stage to find more candidate hints to improve all types of queries.

Chapter 7

Conclusion

The system pipeline has shown promising results in improving query performance which can be attributed to the targeted localized hint generation by our model. By redefining the problem as a sequence generation task to generate hints to optimize every node operation, we overcome the limitations of previous works which are constrained to choosing blanket hints that apply to the entire query due to high computation cost. This highlights, the potential of LLMs to outperform traditional learning-based methods in query optimization tasks. However, as mentioned earlier, further exploration is necessary during the zero-shot pipeline phase to create a more diverse demonstration pool and to further improve query execution times.

Chapter 8

Challenges Faced

Query optimization remains a multifaceted and inherently complex problem, which presented numerous challenges during the process of building our model. Some of these challenges pertain to the inconsistencies in responses from the LLM while others can be attributed to resource optimization. Below we discuss all the challenges faced during the implementation of this project and how we solved them:

1. **LLM Hallucination**: During the initial development phase of the zero-shot pipeline we often encountered erroneous responses from the LLM. To solve this issue, we iteratively modified prompts until we managed to make such errors negligible. during this process we observed that providing the various possible hint formats as options in the prompt and then asking the LLM choose from them significantly reduce hallucination compared to simply asking the LLM to generate the hints using a given format. Another issue we encountered was that the LLM sometimes chose index scan on columns which did not have an index. This could likely be the result of the length of the prompt which could 'confuse' the LLM. Thus to solve this, we repeatedly include the instructions on index scan selection throughout the prompt to ensure the model focuses on it.

2. **Lengthy node embedding times**: The process of creating node feature was initially considerably time consuming as it involved generating BERT embeddings of multiple features, ratio computations and transformation via PCA. doing it for every node added significant overhead in the query embedding process. To solve this we modified the computation methods to use vectors for parallel computation of all nodes at the same

time, significantly reducing the embedding time. Moreover, we also included batch processing of the vectors so that multiple queries could be embedded at the beforehand during training to reduce training time.

3. **High training time**: High training time is a common bottleneck for all learning-based query optimization solutions. In our implementation it primarily occurred due to:

    - large query execution times for some queries.
    - Testing multiple alternate queries for each query to find the ones that actually reduce training time.

    To deal with the above issue we implemented a multi-threading solution using python `multiprocessing` module. All the queries were run simultaneously on the database using multiple threads to add parallelization and reduce training time. Furthermore, a separate thread was created to monitor query execution times of the queries. Any query / modified query which ran for longer than the threshold time(set at 5 minutes for our application) would be automatically terminated by the thread and ignored. The above optimization helped us to significantly reduce our expected training time from a month to under a week.

```
def compare_n_queries(self, queries, num_queries : int = 1, num_runs : int
    = 1):
    # Execute queries in parallel
    with ThreadPoolExecutor(max_workers=num_queries) as executor:
        futures = [executor.submit(self.runExecutions, queries[i], num_runs
    ) for i in range(num_queries)]

        # Collect execution times
        results = [future.result() for future in futures]
    improved_list = []
    alt_query_time_list = []
    query_result = results.pop(0)
    if query_result[1] is not None:
        return None, None, None, query_result[1]
    else:
        query_avg = query_result[0]
    for result in results:
        if result[1] is not None:
```

```
            improved_list.append(False)
            alt_query_time_list.append(None)
            continue
        alt_query_time = result[0]
        alt_query_time_list.append(alt_query_time)
        improved = alt_query_time < query_avg
        improved_list.append(improved)

    return (query_avg, improved_list, alt_query_time_list, None)
```

Code Snippet 8.1: multithreading of queries

```
def monitor_long_queries(self):
    """
    Periodically checks for long-running queries and cancels them.
    """
    while not self.monitor_thread_stop_event.is_set():
        try:
            conn = psycopg2.connect(
            dbname=self.db_name,
            user=self.db_user,
            password=self.db_password,
            host=self.db_host,
            port=self.db_port
        )
            conn.autocommit = True
            cur = conn.cursor()

            # Query to find long-running queries
            cur.execute("""
                SELECT pid, age(now(), query_start), query
                FROM pg_stat_activity
                WHERE state = 'active'
                AND query_start < now() - interval '%s seconds';
            """, (self.query_timeout,))

            long_queries = cur.fetchall()

            for pid, duration, query in long_queries:
                print(f"Cancelling query (PID: {pid}, Duration: {duration})
```

```
    : {query}")
29                  cur.execute(f"SELECT pg_cancel_backend({pid})")
30
31              cur.close()
32              conn.close()
33
34          except Exception as e:
35              print(f"Error monitoring queries: {e}")
36
37          # Check every 10 seconds
38          time.sleep(10)
39      print('exiting the monitor query thread')
```

Code Snippet 8.2: thread for monitoring long queries

Chapter 9

Limitations and Future Work

Due to resource and time constraints the project faced certain limitations. Specifically, queries with exceptionally long execution times (exceeding 5 minutes) were excluded from the training process, as including them would have substantially increased training time and hindered the timely completion of the project. Moreover, in our experiments we only generated 3 different hintset combinations in the zero-shot pipeline. By including more hintsets, we could increase the diversity of the demonstration pool and learn better hints, boosting the performance of our system.

In addition to the above limitations, there are quite some areas for improvement that we identified but were beyond the scope of this project. Including these implementations could add to the effectivity and novelty of our solution.

1. **PostgreSQL Integration**: The current system is experimental. It could benefit by creating a module to provide PostgreSQL integration so that our query optimizer can be added as an extension, providing native support. This would void the need for running the source code and executing your queries through python as users could use it as an inbuilt feature in PostgreSQL

2. **Improving the Query Embedding module**: Through the course of the project, I realized that query embedding in itself requires significant more work. While our method provides a generalizable vector representation, there are multiple aspects that are yet unexplored. For instance, methods for embedding the relation data itself and adding those embeddings as leaf nodes to the scan nodes could produce more robust representations. Other methods

such as masking relation, and column names to provide generalization across multiple databases are also worth exploring. Several additional such factors were identified that could be incorporated and explored in future work to evaluate their influence on the efficiency of downstream tasks. However, experimenting with all these factors would be very time-consuming, and hence were omitted from the scope of the project.

3. **Join reordering**: Lastly, for queries with multiple joins, accurately reordering the joins can help significantly reduce the query execution time. However, as this field, requires extensive work of its own, it was beyond the time constraints of this project, and hence was not included.

Chapter 10

Appendix

10.1 Parsing the Query Tree

```
def parseQep(self, qep):
    """
    Parse the query execution plan (QEP) and build the nodes and edges for
    the graph.
    """

    def traversePlan(plan_tree, parent_id=None):
        node_id = self.node_counter  # Use the counter as the node ID
        self.node_counter += 1
        node_label = plan_tree['Node Type']
        total_cost = plan_tree.get('Total Cost', -1)
        rows = plan_tree.get('Plan Rows', -1)
        rel_name = plan_tree.get('Relation Name', None)
        index_name = plan_tree.get('Index Name', None)
        node_filter = plan_tree.get('Filter', None)
        join_filter = plan_tree.get('Join Filter', None)
        hash_cond = plan_tree.get('Hash Cond', None)
        merge_cond = plan_tree.get('Merge Cond', None)
        index_cond = plan_tree.get('Index Cond', None)
        width = plan_tree.get('Plan Width', 0)
        parent_hash = False
        if parent_id != None:
            if self.nodes[parent_id]['label'] == 'Hash':
```

```
                parent_hash = True

        node_info = {
            'parent id' : parent_id,
            'id': node_id,
            'label': node_label,
            'Cummulative Cost': total_cost,
            'Planned rows': rows,
            'Relation Name': rel_name,
            'Index Name': index_name,
            'Filter' : node_filter if node_filter else join_filter,
            'Join Condition' : hash_cond if hash_cond is not None else
    merge_cond,
            'Index Condition' : index_cond,
            'Width' : width,
            'Parent Hash': parent_hash
        }
        self.nodes.append(node_info)

        if parent_id is not None:
            self.edges[0].append(node_id)
            self.edges[1].append(parent_id)

        if 'Plans' in plan_tree:
            for subplan in plan_tree['Plans']:
                # Recursively traverse the plan trees
                traversePlan(subplan, parent_id=node_id)

    # Start traversing from the root plan
    root_plan = qep[0]['Plan']
    # Traverse the plan tree
    traversePlan(root_plan)
```

Code Snippet 10.1: Recursively parsing the Query tree

10.2 GNN Module

```
class GNNEncoder(torch.nn.Module):
```

```
    def __init__(self, input_dim, hidden_dim, output_dim, pe_dim):
        super(GNNEncoder, self).__init__()
        self.input_dim = input_dim
        self.hidden_dim = hidden_dim
        self.output_dim = output_dim
        self.pe_dim = pe_dim
        self.conv1 = GCNConv(input_dim + pe_dim, hidden_dim)
        self.conv2 = GCNConv(hidden_dim, output_dim)

    def forward(self, data : Data, batch):

        def normalize_features(data):
            """ Normalize data.x using Z-score normalization. """
            mean = data.x.mean(dim=0, keepdim=True)
            std = data.x.std(dim=0, keepdim=True) + 1e-6
            data.x = (data.x - mean) / std
            return data

        data = normalize_features(data)
        num_nodes = data.x.shape[0]

        k_adjusted = min(num_nodes, self.pe_dim)

        pos_encoder = AddLaplacianEigenvectorPE(k=k_adjusted, attr_name='
pos_enc')
        data = pos_encoder(data=data)

        if k_adjusted < self.pe_dim:
            padding = torch.zeros((num_nodes, self.pe_dim - k_adjusted),
device=data.x.device)
            data.pos_enc = torch.cat([data.pos_enc, padding], dim=1)

        data.pos_enc = data.pos_enc.to(data.x.device)

        x, edge_index = data.x, data.edge_index
        x = torch.cat([x, data.pos_enc], dim=1)

        x = self.conv1(x, edge_index)
        x = F.relu(x)
```

```
        x = self.conv2(x, edge_index)

        x = global_add_pool(x, batch)
        return x
```

Code Snippet 10.2: GNNEncoder with Laplacian Positional Encoding

Bibliography

[1] Aug. 2023. URL: https://docs.oracle.com/en/database/oracle/oracle-database/21/tgsql/query-optimizer-concepts.html#GUID-06129ACE-36B2-4534-AE68-EDFCAEBB3B5D.

[2] PostgreSQL. *50.5. Planner/Optimizer*. https://www.postgresql.org/docs/current/planner-optimizer.html. PostgreSQL Documentation, Accessed: 2025-01-26. Nov. 2024.

[3] Viktor Leis et al. "How Good Are Query Optimizers, Really?" In: *VLDB* 9.3 (2015), pp. 204–215.

[4] SIGMOD Blog. *IS QUERY OPTIMIZATION A "SOLVED" PROBLEM?* ACM SIGMOD Blog. Accessed: 2025-01-26. 2014. URL: http://wp.sigmod.org/?p=1075.

[5] Hai Lan, Zhifeng Bao, and Yuwei Peng. "A Survey on Advancing the DBMS Query Optimizer: Cardinality Estimation, Cost Model, and Plan Enumeration". In: *Data Science and Engineering* 6.1 (Mar. 2021), pp. 86–101. ISSN: 2364-1541. DOI: 10.1007/s41019-020-00149-7. URL: https://doi.org/10.1007/s41019-020-00149-7.

[6] Yuxing Han et al. *Cardinality Estimation in DBMS: A Comprehensive Benchmark Evaluation*. 2021. arXiv: 2109.05877 [cs.DB]. URL: https://arxiv.org/abs/2109.05877.

[7] *Row Estimation Examples*. Nov. 2024. URL: https://www.postgresql.org/docs/current/row-estimation-examples.html.

[8] MikeRayMSFT. *Cardinality Estimation (SQL Server) - SQL server*. URL: https://learn.microsoft.com/en-us/sql/relational-databases/performance/cardinality-estimation-sql-server?view=sql-server-ver16.

[9] *Configuring Persistent Optimizer Statistics Parameters*. URL: https://dev.mysql.com/doc/refman/8.4/en/innodb-persistent-stats.html.

[10] Kostas Tzoumas, Amol Deshpande, and Christian S. Jensen. “Lightweight Graphical Models for Selectivity Estimation Without Independence Assumptions”. In: *VLDB* 4.11 (2011), pp. 852–863.

[11] Kostas Tzoumas, Amol Deshpande, and Christian S. Jensen. “Efficiently Adapting Graphical Models for Selectivity Estimation”. In: *VLDB Journal* 22.1 (2013), pp. 3–27.

[12] Martin Halford, Pierre Saint-Pierre, and Franck Morvan. “An Approach Based on Bayesian Networks for Query Selectivity Estimation”. In: *Proceedings of DAS-FAA*. 2019, pp. 3–19.

[13] Ryan Marcus and Olga Papaemmanouil. “Deep Reinforcement Learning for Join Order Enumeration”. In: *Proceedings of the First International Workshop on Exploiting Artificial Intelligence Techniques for Data Management*. SIGMOD/PODS ’18. ACM, June 2018. DOI: 10.1145/3211954.3211957. URL: http://dx.doi.org/10.1145/3211954.3211957.

[14] Ryan Marcus et al. “Neo: a learned query optimizer”. In: *Proceedings of the VLDB Endowment* 12.11 (July 2019), pp. 1705–1718. ISSN: 2150-8097. DOI: 10.14778/3342263.3342644. URL: http://dx.doi.org/10.14778/3342263.3342644.

[15] Ji Sun and Guoliang Li. *An End-to-End Learning-based Cost Estimator*. 2019. arXiv: 1906.02560 [cs.DB]. URL: https://arxiv.org/abs/1906.02560.

[16] Andreas Kipf et al. *Learned Cardinalities: Estimating Correlated Joins with Deep Learning*. 2018. arXiv: 1809.00677 [cs.DB]. URL: https://arxiv.org/abs/1809.00677.

[17] Andreas Kipf et al. *Estimating Cardinalities with Deep Sketches*. 2019. arXiv: 1904.08223 [cs.DB]. URL: https://arxiv.org/abs/1904.08223.

[18] Jennifer Ortiz et al. *Learning State Representations for Query Optimization with Deep Reinforcement Learning*. 2018. arXiv: 1803.08604 [cs.DB]. URL: https://arxiv.org/abs/1803.08604.

[19] Immanuel Trummer et al. “SkinnerDB: Regret-Bounded Query Evaluation via Reinforcement Learning”. In: *Proceedings of the 2019 International Conference on Management of Data*. SIGMOD/PODS ’19. ACM, June 2019. DOI: 10.1145/3299869.3300088. URL: http://dx.doi.org/10.1145/3299869.3300088.

[20] Ryan Marcus et al. “Bao: Making Learned Query Optimization Practical”. In: *Proceedings of the 2021 International Conference on Management of Data*. SIGMOD ’21.

Virtual Event, China: Association for Computing Machinery, 2021, pp. 1275–1288. ISBN: 9781450383431. DOI: 10.1145/3448016.3452838. URL: https://doi.org/10.1145/3448016.3452838.

[21] Zhaodonghui Li et al. *LLM-R2: A Large Language Model Enhanced Rule-based Rewrite System for Boosting Query Efficiency*. 2024. arXiv: 2404.12872 [cs.DB]. URL: https://arxiv.org/abs/2404.12872.

[22] *Hint list*. URL: https://pg-hint-plan.readthedocs.io/en/latest/hint_list.html.

[23] Lucas Woltmann et al. "FASTgres: Making Learned Query Optimizer Hinting Effective". In: *Proc. VLDB Endow.* 16.11 (July 2023), pp. 3310–3322. ISSN: 2150-8097. DOI: 10.14778/3611479.3611528. URL: https://doi.org/10.14778/3611479.3611528.

[24] Christoph Anneser et al. "AutoSteer: Learned Query Optimization for Any SQL Database". In: *Proc. VLDB Endow.* 16.12 (Aug. 2023), pp. 3515–3527. ISSN: 2150-8097. DOI: 10.14778/3611540.3611544. URL: https://doi.org/10.14778/3611540.3611544.

[25] Luming Sun et al. "DeepO: A Learned Query Optimizer". In: *Proceedings of the 2022 International Conference on Management of Data*. SIGMOD '22. Philadelphia, PA, USA: Association for Computing Machinery, 2022, pp. 2421–2424. ISBN: 9781450392495. DOI: 10.1145/3514221.3520167. URL: https://doi.org/10.1145/3514221.3520167.

[26] Sergey Zinchenko and Sergey Iazov. *HERO: Hint-Based Efficient and Reliable Query Optimizer*. 2024. arXiv: 2412.02372 [cs.DB]. URL: https://arxiv.org/abs/2412.02372.

[27] Peter Akioyamen, Zixuan Yi, and Ryan Marcus. *The Unreasonable Effectiveness of LLMs for Query Optimization*. 2024. arXiv: 2411.02862 [cs.DB]. URL: https://arxiv.org/abs/2411.02862.

[28] Ryan Marcus and Olga Papaemmanouil. "Plan-structured deep neural network models for query performance prediction". In: *Proceedings of the VLDB Endowment* 12.11 (July 2019), pp. 1733–1746. ISSN: 2150-8097. DOI: 10.14778/3342263.3342646. URL: http://dx.doi.org/10.14778/3342263.3342646.

[29] Ji Sun and Guoliang Li. "An end-to-end learning-based cost estimator". In: *Proc. VLDB Endow.* 13.3 (Nov. 2019), pp. 307–319. ISSN: 2150-8097. DOI: 10.14778/3368289.3368296. URL: https://doi.org/10.14778/3368289.3368296.

[30] Yue Zhao et al. “QueryFormer: a tree transformer model for query plan representation”. In: *Proc. VLDB Endow.* 15.8 (Apr. 2022), pp. 1658–1670. ISSN: 2150-8097. DOI: 10.14778/3529337.3529349. URL: https://doi.org/10.14778/3529337.3529349.

[31] Thomas N. Kipf and Max Welling. *Semi-Supervised Classification with Graph Convolutional Networks*. 2017. arXiv: 1609.02907 [cs.LG]. URL: https://arxiv.org/abs/1609.02907.

[32] Vijay Prakash Dwivedi et al. *Benchmarking Graph Neural Networks*. 2022. arXiv: 2003.00982 [cs.LG]. URL: https://arxiv.org/abs/2003.00982.

[33] Ting Chen et al. *A Simple Framework for Contrastive Learning of Visual Representations*. 2020. arXiv: 2002.05709 [cs.LG]. URL: https://arxiv.org/abs/2002.05709.

[34] Fan-Yun Sun et al. *InfoGraph: Unsupervised and Semi-supervised Graph-Level Representation Learning via Mutual Information Maximization*. 2020. arXiv: 1908.01000 [cs.LG]. URL: https://arxiv.org/abs/1908.01000.

[35] Yuning You et al. *Graph Contrastive Learning with Augmentations*. 2021. arXiv: 2010.13902 [cs.LG]. URL: https://arxiv.org/abs/2010.13902.

[36] Kaveh Hassani and Amir Hosein Khasahmadi. *Contrastive Multi-View Representation Learning on Graphs*. 2020. arXiv: 2006.05582 [cs.LG]. URL: https://arxiv.org/abs/2006.05582.